COMP 4330SEF : Advanced Programming and AI Algorithms

Evaluating CPU Inference Performance of scikit-learn and ONNX Runtime

Lam Yu Kwan (Leader) 13487539

Tsang Yik Long 13487503

To Chung Hei 13493141

Siu Jyut 13483577

RAWAT Aaditya 13231466

P01 Group 2

MOTIVATION & PROBLEM

Scikit-learn is convenient for training but slow and resource-intensive in production inference, while converting models to ONNX can improve performance. The core challenge remains ensuring prediction consistency with the original outputs, especially regarding ZipMap behavior and floating-point differences.

---

1) Speed Gap
How large is the difference in CPU inference time and throughput?

2) ZipMap Cost
What is the performance & numerical impact of ZipMap on/off?

3) Batch Scaling
How does behavior change across different batch sizes ?

Conduct controlled experiments to quantify the actual trade-offs between scikit-learn and ONNX Runtime.

METHODOLOGY OVERVIEW

SETUP

Model : Random Forest (10 / 100 /500 trees)

Data : Breast Cancer Wisconsin · 569 × 30

Split : Fixed 80/20 · seed 42

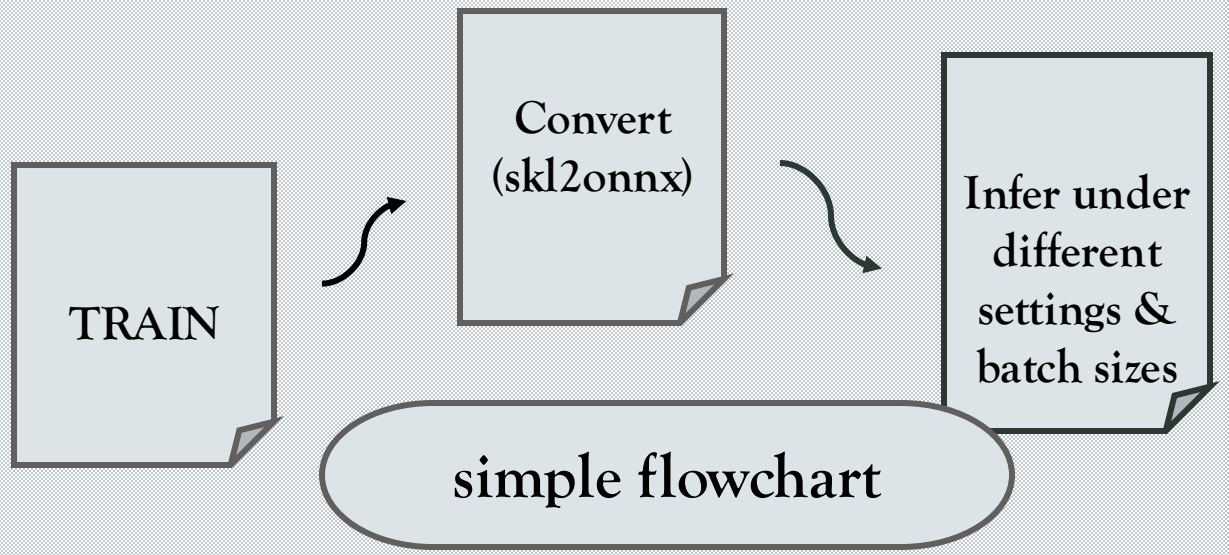
Hardware : AMD + Intel CPUs · single thread

Timing : 50 warm-ups → 1,000 timed calls

THREE PATHS COMPARED

- scikit-learn predict_proba()
- ONNX Runtime + ZipMap enabled
- ONNX Runtime + ZipMap disabled

Batches tested: 1 · 16 · 64 ·256



EVALUATION METRICS

Inference time : ms per batch call

Throughput : rows processed / second

Label agreement : ONNX vs scikit-learn class

Probability metrics : max / mean abs. difference

EXPERIMENTAL SETUP & KEY DESIGN CHOICES

Why Random Forest + this dataset

- Simple and fully reproducible
- Tree-based models stress the conversion path
- Clear ground truth for label agreement

Why ZipMap is important

- Controls output format (map vs matrix)
- Directly affects latency & post-processing
- Common source of numerical differences

Why we vary batch size

- Batch = 1 ≈ single real-time request
- Batch = 64 / 256 ≈ typical service batching
- Reveals scaling behaviour of each runtime

Controlled variables

- Same trained model
- Same hardware (CPU only)
- Same fixed inputs every time

MAIN RESULTS

ONNX was faster on both computers

100-tree forest, ONNX matrix output, milliseconds per batch

Batch	AMD ms	AMD speedup	Intel ms range	Intel speedup range
1	0.00970	482.93x	0.00650 - 0.00660	393.40 - 402.05x
16	0.01660	157.48x	0.01960 - 0.01970	133.04 - 134.94x
64	0.07620	42.41x	0.11070 - 0.11395	24.34 - 24.79x
256	0.64045	9.95x	0.43570 - 0.44090	6.95 - 6.98x

Speedup = scikit-learn latency / ONNX Runtime latency

AMD batch 1: 4.6844 ms in sklearn, 0.00970 ms in ONNX

CPU, Windows and Python differed together. Intel values span three repeats.

Source: Final English Report, pp. 6-7 06

AGREEMENT & DIFFERENCE

100%

Label agreement
(all 10 / 100 / 500 trees)

Max abs. probability difference

< 1e-5

Never changes the final class label

INFERENCE TIME & SPEEDUP

Batch	ONNX (ms)	sklearn (ms)	Speedup
1	0.0097	4.68	483×
16	0.0166	2.61	157×
64	0.0762	3.23	42×
256	0.640	6.37	10×

TAKEAWAY :

At every tested batch, ONNX (ZipMap off) is clearly faster than scikit-learn.
Label predictions stay identical; probability differences stay below tolerance.

ANALYSIS & INSIGHTS

Cost of ZipMap

- Extra processing increases latency
- Especially visible at small batches
- Faster in 35 of 36 pairs when off

Causal explanation

- Output format: dict-like vs flat array
- Directly affects memory access
- And post-processing cost

When to keep / turn off

- Need labelled map → keep ZipMap on
- Only need matrix / labels → turn off
- Most service scenarios should prefer off

Practical meaning

- Lightweight web services
- Edge deployment
- Prefer ZipMap-off + proper batching

CONCLUSION + FUTURE WORK

1. In CPU inference experiments, ONNX showed greater relative speedups over scikit-learn at smaller batch sizes.

   approximately 1,052× speedup at batch 1 and 7× at batch 256.

2. Predictions stay consistent (100% label agreement), while probability differences remain measurable but tiny.

3. Conversion is not a free lunch — it must be tuned to the target application scenario.

Future Work

Larger models · GPU inference



THANK YOU

Q&A